

1.ÜNİTE - C# ile Görsel Programlamaya Giriş

Geleneksel konsol uygulamaları ile modern grafiksel kullanıcı arayüzleri (GUI) arasındaki temel farkları, .NET platformunda kullanılan WinForms ve WPF gibi arayüz çerçevelerini ve görsel programlama için temel geliştirme ortamı olan Visual Studio'nun kurulum ve kullanım detaylarını ele almaktadır.

1. Konsol ve Grafiksel Arayüz Karşılaştırması

Geleneksel terminal tabanlı uygulamalar ile modern görsel yazılımlar arasındaki temel farklar şunlardır:

- **Konsol Uygulamaları:** Sadece metin tabanlı etkileşim sunar. Sistem kaynaklarını minimum düzeyde tüketir ancak komutların ve parametrelerin ezberlenmesini gerektirdiği için son kullanıcı için zordur. Genellikle aynı anda tek bir işlem yürütülebilir.
- **Grafiksel Kullanıcı Arayüzü (GUI):** Formlar ve görsel bileşenler (butonlar, metin kutuları vb.) aracılığıyla kullanıcıyla etkileşime girer. Daha kullanıcı dostudur ve hataya daha az sebebiyet verir. Modern yazılımların büyük çoğunluğu bu yapıdadır.

2. .NET Grafiksel Arayüz Çerçeveleri

C# diliyle Windows üzerinde görsel uygulama geliştirmek için iki ana teknoloji öne çıkar:

- **Windows Forms (WinForms):**
 - Sürükle-bırak yöntemiyle hızlı ve kolay tasarım imkânı sunar.
 - Windows API'lerini kullanır ve daha az bellek tüketir.
 - Küçük ölçekli projeler ve öğrenme aşaması için idealdir.
 - Grafik ve animasyon kalitesi daha düşüktür; büyük projelerde yönetimi zorlaşabilir.
- **Windows Presentation Foundation (WPF):**
 - DirectX tabanlıdır ve donanım hızlandırma kullanarak yüksek kaliteli grafikler sunar.

- Tasarım ve kodun ayrılması için **XAML** işaretleme dilini kullanır.
- Büyük ölçekli ve kurumsal uygulamalar için daha uygundur ancak öğrenme süreci daha uzundur.

3. Visual Studio ve Geliştirme Ortamı

Görsel programlama sürecinde en yaygın kullanılan editör Visual Studio'dur. Programın kurulumu sırasında ".NET masaüstü geliştirme" iş yükünün seçilmesi kritik öneme sahiptir.

- **Proje Yapısı:** En geniş kapsamlı yapı "**Çözüm**" (**Solution**) olarak adlandırılır ve içerisinde bir veya birden fazla "**Proje**" barındırabilir.
- **Kısmî Sınıf (Partial Class):** C#'ta bir sınıfın kodları birden fazla dosyaya ayrılabilir. WinForms'ta form kodları `Form1.cs` (olaylar ve metodlar) ve `Form1.Designer.cs` (görsel bileşen kodları) olarak ikiye ayrılır.

4. Visual Studio Temel Panelleri

Görsel programlama yaparken en sık kullanılan paneller ve işlevleri:

- **Tasarım Ekranı:** Formun görsel olarak tasarlandığı ana alandır.
- **Araç Kutusu (Toolbox):** Form üzerine eklenecek buton, etiket gibi bileşenlerin bulunduğu paneldir.
- **Çözüm Gezgini (Solution Explorer):** Proje dosyalarının, formların ve kaynak kodların listelendiği yönetim panelidir.
- **Özellikler Paneli (Properties):** Seçili olan formun veya bileşenin adını, rengini, metnini ve diğer özelliklerini değiştirmek için kullanılır.
- **Hata Listesi:** Kod yazımı sırasında oluşan yazım ve mantık hatalarını gösteren paneldir.

2.ÜNİTE - FORMLAR

Formlar, bir Windows masaüstü uygulamasının kullanıcıya görünen yüzüdür. Tüm görsel kontroller (butonlar, metin kutuları vb.) form üzerine yerleştirilir ve formlar aracılığıyla kullanıcıyla etkileşim kurulur. Bir uygulama tek bir formdan (SDI) oluşabileceği gibi, birden fazla formun (MDI) bir arada kullanıldığı karmaşık yapılara da sahip olabilir.

Form Özellikleri (Properties)

Formun görünümünü ve davranışını belirleyen ayarlardır. Bu ayarlar Visual Studio'daki **Özellikler** panelinden veya doğrudan kod içerisinde değiştirilebilir.

- **Name:** Formun kod tarafında çağrılacağı benzersiz isimdir.
- **Text:** Formun başlık çubuğunda görünen metindir.
- **BackColor & BackgroundImage:** Formun arka plan rengini veya resmini belirler.
- **FormBorderStyle:** Pencerenin kenarlık stilini (boyutlandırılabilir, sabit vb.) belirler.
- **ControlBox:** Sağ üstteki simge durumuna küçült, tam ekran ve kapat butonlarının görünürlüğünü kontrol eder.
- **Opacity:** Formun saydamlık derecesini ayarlar.
- **StartPosition:** Formun ekranın neresinde (örneğin merkezde) başlayacağını belirler.
- **TopMost:** Formun her zaman diğer pencerelerin üstünde kalmasını sağlar.

Form Olayları (Events)

Kullanıcının form üzerinde yaptığı işlemlere (tıklama, kapatma, taşıma vb.) programın vereceği tepkilerdir.

- **Load:** Form hafızaya yüklenip ilk kez gösterilmeden hemen önce tetiklenir.
- **Shown:** Form kullanıcıya tamamen gösterildiğinde tetiklenir.
- **FormClosing:** Form henüz kapanma aşamasındayken (iptal edilebilir durumda) tetiklenir.
- **FormClosed:** Kapatma işlemi bittikten sonra tetiklenir.
- **Click & DoubleClick:** Forma tek veya çift tıklandığında gerçekleşir.
- **Resize:** Formun boyutu değiştirildiğinde çalışır.

Formlar Arası Geçiş ve Yönetim

Uygulama içerisinde birden fazla form kullanıldığında, bu formlar arasındaki etkileşim belirli metodlarla sağlanır.

- **Show():** Belirtilen formu açar; ana form hala tıklanabilir kalır.
- **ShowDialog():** Formu "modal" olarak açar; bu form kapatılmadan ana forma dönülemez.
- **Hide():** Formu kapatmaz, sadece görünmez yapar.
- **Close():** Formu kapatır ve işletim sistemine kapatma emri gönderir.
- **Dispose():** Formun kullandığı sistem kaynaklarını tamamen serbest bırakır.
- **Application.Run():** Uygulamanın hangi form ile başlayacağını belirler (genellikle Program.cs içindedir).

Kontrol Ekleme ve Silme

Form üzerine görsel öğeler eklemek için üç temel yol vardır:

- **Araç Kutusu (Toolbox):** İstenilen kontrole çift tıklanarak veya sürükle-bırak yöntemiyle eklenir.
- **Kod ile Ekleme:** `Controls.Add()` komutu kullanılarak çalışma anında nesne oluşturulabilir.
- **Silme:** Tasarım ekranında nesne seçilip **Delete** tuşuna basılarak veya sağ tıklayıp **Sil** denilerek kaldırılır.

3.ÜNİTE - FORM KONTROLLERİ

.NET ortamında zengin grafiksel arayüzler tasarlamak için kullanılan temel form kontrollerini ele almaktadır. Kontroller, işlevlerine göre Araç Kutusu'nda (Toolbox) gruplandırılmış olup sürükle-bırak yöntemiyle forma eklenebilir.

1. Temel Giriş ve Çıkış Kontrolleri

Kullanıcıdan veri almak veya bilgi sunmak için kullanılan en temel bileşenlerdir.

- **Button (Düğme):** Bir eylemi başlatmak için kullanılır. Tıklama (Click) ve çift tıklama (DoubleClick) en yaygın olaylarıdır. Text özelliği butondaki metni, BackColor ise rengini belirler.

- **TextBox:** Kullanıcıdan düz metin verisi almak için kullanılır. Text özelliği ile veri okunur veya atanır. Multiline özelliği çok satırlı girişe, PasswordChar ise parolaların maskelenmesine imkân tanır.
- **RichTextBox:** TextBox'tan farklı olarak biçimlendirilmiş metinleri (RTF), resim ve tabloları destekler. Metnin font, renk ve biçim gibi zengin özelliklerini işleyebilir.
- **Label (Etiket):** Diğer kontrollere açıklayıcı metin sağlamak için kullanılır. Kullanıcı etkileşimi zayıftır, genellikle sadece bilgi sunar.

2. Seçim Kontrolleri

Kullanıcıya sunulan seçenekler arasından seçim yapılmasını sağlayan kontrollerdir.

- **RadioButton:** Birden fazla seçenek arasından **yalnızca bir tanesinin** seçilmesine izin verir. Bir kutucuk seçildiğinde diğerinin seçimi otomatik olarak kalkar. Seçim durumu Checked özelliğiyle kontrol edilir.
- **CheckBox (Onay Kutusu):** Bir veya daha fazla seçeneğin aynı anda işaretlenmesine imkân tanır. RadioButton'un aksine bağımsız seçimler yapılabilir.
- **GroupBox:** RadioButton veya CheckBox gibi kontrolleri kategorilere ayırarak gruplandırmak için kullanılır.

3. Liste Kontrolleri

Verileri bir liste halinde sunan veya kullanıcıya bu listeden seçim yaptıran bileşenlerdir.

- **ListBox:** Elemanları bir liste halinde sunar. Add () ile eleman eklenir, Remove () veya RemoveAt () ile silinir. SelectedIndex özelliği seçili elemanın sırasını verir.
- **CheckedListBox:** ListBox ve CheckBox kontrollerinin birleşimidir; listedeki her elemanın yanında bir onay kutusu bulunur.
- **ComboBox (Açılır Liste):** Seçenekleri açılır bir liste olarak sunar. DropDownStyle özelliği listenin davranışını belirler:
 - **Simple:** Elemanları liste olarak gösterir ve veri girişine izin verir.
 - **DropDown:** Açılır liste sunar ve dışarıdan veri girişine izin verir (varsayılan).

- **DropDownList:** Sadece listedeki elemanların seçilmesine izin verir, dışarıdan veri girilemez.

Önemli Metodlar ve Farklar

- **Metod vs Olay:** Metod bir kontrolün kendi içindeki eylemleridir (Örn: `Focus()`, `Clear()`); olay ise kullanıcının etkileşimidir (Örn: `Click`).
- **Veri Dönüşümü:** `TextBox` veriyi her zaman `string` olarak alır. Sayısal işlemler yapılacaksa `Convert.ToDouble()` gibi metodlarla dönüştürülmelidir.
- **Focus():** İmleci ilgili kontrole odaklamak ve kullanıcıyı o alana yönlendirmek için kullanılır.

4.ÜNİTE - GÖRÜNÜM VE DİYALOG KONTROLLERİ

Bir yazılımın profesyonel bir görünüme kavuşmasını sağlayan menü sistemleri, form üzerindeki karmaşıklığı azaltan gruplandırma elemanları ve dosya açma/kaydetme gibi işletim sistemi düzeyindeki standart diyalogların kullanımı incelenmektedir.

1. Menü ve Durum Şeritleri

Kullanıcının uygulama komutlarına hızlıca erişmesini ve uygulamanın anlık durumunu takip etmesini sağlayan yapılardır.

- **MenuStrip:** Formun üst kısmında hiyerarşik bir menü yapısı oluşturur (Dosya, Düzen vb.). Alt menüler ve kısa yollar eklenebilir.
- **ContextMenuStrip:** Belirli bir kontrole sağ tıklandığında açılan "bağlam menüsü"dür. Kontrolün `ContextMenuStrip` özelliğiyle ilişkilendirilir.
- **StatusStrip:** Formun en alt kısmında yer alır. Uygulama durumu, işlem süreci (`ProgressBar` ile) veya kullanıcı bilgisi gibi verileri göstermek için kullanılır.

2. Kapsayıcı (Container) Kontrolleri

Arayüz bileşenlerini mantıksal veya görsel olarak gruplandırmak ve form boyutlandığında yerleşimlerini yönetmek için kullanılırlar.

- **Panel:** Diğer kontrolleri bir arada tutan basit bir çerçevedir. `AutoScroll` özelliği ile içerik panele sığmadığında kaydırma çubukları oluşturabilir.
- **FlowLayoutPanel:** İçindeki kontrolleri bir akış yönüne (soldan sağa veya yukarıdan aşağıya) göre otomatik olarak sıralar.
- **TabControl:** Formu sekmelere ayırarak sınırlı alanda çok fazla içeriğin gösterilmesini sağlar. Her sekme (`TabPage`) bağımsız bir alan sunar.
- **SplitContainer:** Formu iki hareketli bölmeye ayırır. Kullanıcı bölmeler arasındaki sınırı sürükleyerek alanların boyutunu değiştirebilir.

3. Standart Diyalog Pencereleeri (Common Dialogs)

Windows işletim sisteminin sunduğu standart etkileşim pencereleridir. Bu kontroller formun tasarım ekranında görünmez, çalışma anında çağırılırlar.

- **OpenFileDialog:** Kullanıcının bilgisayarından bir veya birden fazla dosya seçmesini sağlar.
- **SaveFileDialog:** Verilerin bir dosyaya kaydedilmesi için konum ve dosya adı seçtirir.
- **ColorDialog:** Kullanıcıya bir renk paleti sunarak renk seçimi yapmasına olanak tanır.
- **FontDialog:** Yazı tipi, boyutu ve stili seçimi için kullanılır.

4. Yazdırma Kontrolleri

Uygulama içerisindeki verilerin kağıda aktarılması süreçlerini yönetir.

- **PrintDocument:** Yazdırılacak içeriğin tanımını yapar.
- **PrintPreviewDialog:** Yazdırma işlemi öncesinde belgenin nasıl görüneceğine dair bir önizleme sunar.
- **PrintDialog:** Yazıcı seçimi ve sayfa ayarları gibi temel yazdırma parametrelerini yönetir.

5.ÜNİTE - YARDIMCI FORM KONTROLLERİ

Standart metin kutularının ötesinde, belirli veri türlerini (tarih, sayı, resim vb.) yönetmek ve kullanıcıya işlemin gidişatı hakkında görsel geri bildirim vermek için kullanılan özel kontroller ele alınmıştır.

1. Sayısal ve Maskeli Giriş Kontrolleri

Kullanıcının girdiği veriyi belirli bir formatta veya aralıkta tutmak için kullanılırlar.

- **NumericUpDown:** Kullanıcının sadece belirli bir aralıktaki sayısal değerleri girmesini sağlar. Minimum ve Maximum özellikleriyle sınırlandırılabilir; Value özelliği ile güncel sayısal değer alınır.
- **MaskedTextBox:** Telefon numarası, T.C. kimlik numarası veya tarih gibi belirli bir kalıba (maskeye) uygun veri girişi yaptırmak için kullanılır. Bu sayede hatalı veri girişinin önüne geçilir.

2. Tarih ve Zaman Kontrolleri

Uygulamalarda tarihsel verilerin hatasız seçilmesini sağlayan bileşenlerdir.

- **DateTimePicker:** Kullanıcıya açılır bir takvim sunarak tek bir tarih veya saat seçtirir. Seçilen değer Value özelliğiyle alınırken, Format özelliğiyle görünüm (Kısa tarih, Uzun tarih vb.) özelleştirilebilir.
- **MonthCalendar:** Form üzerinde doğrudan bir takvim görüntüler ve kullanıcının belirli bir tarih aralığını (başlangıç ve bitiş) seçmesine olanak tanır.

3. Görsel ve Bilgilendirme Kontrolleri

Kullanıcı deneyimini zenginleştiren ve uygulamanın etkileşimini artıran elemanlardır.

- **PictureBox:** Form üzerinde .jpg, .png, .gif gibi formatlardaki resimleri görüntülemek için kullanılır. SizeMode özelliği ile resmin kutu içine nasıl sığdırılacağı (Stretch, Zoom vb.) belirlenir.

- **ProgressBar:** Devam eden bir işlemin (dosya indirme, kaydetme vb.) ne kadarının tamamlandığını görsel bir çubukla gösterir. Value özelliği ilerleme miktarını temsil eder.
- **ImageList:** Birden fazla resmi tek bir koleksiyon içinde saklar. Genellikle butonlara veya menülere ikon sağlamak amacıyla arka planda kullanılır.

4. Diğer Yardımcı Kontroller

- **Timer:** Belirli zaman aralıklarında (milisaniye cinsinden) otomatik olarak işlem yapılmasını sağlar. Interval özelliği süreyi belirlerken, her süre dolusunda Tick olayı tetiklenir.
- **ToolTip:** Fare bir kontrolün üzerine geldiğinde küçük bir yardım baloncucu (açıklama metni) gösterir.
- **NotifyIcon:** Uygulamanın Windows görev çubuğundaki bildirim alanında (saatin yanı) bir simge ile çalışmasını ve bildirimler göstermesini sağlar.
- **WebBrowser:** Form içerisine bir web tarayıcısı ekleyerek internet sayfalarının görüntülenmesine imkân tanır.

6.ÜNİTE - VERİ İŞLEME

Değişkenlerin tanımlanması, veri türleri arasındaki kritik dönüşümler, sabitlerin kullanımı ve verilerin gruplandırılmasını sağlayan diziler ele alınmıştır.

1. Değişkenler ve Sabitler

Verilerin bilgisayar belleğinde geçici olarak saklanmasını sağlayan yapılardır.

- **Değişken Tanımlama:** Veri tipi ve değişken adı belirtilerek yapılır (Örn: `int sayi;`).
- **İsimlendirme Kuralları:** Değişken adları sayı ile başlayamaz, boşluk içeremez ve C# diline özgü anahtar kelimelerden oluşamaz.
- **Sabitler (const):** Program boyunca değeri hiç değişmeyecek veriler için kullanılır (Örn: Pi sayısı).

2. Veri Türleri

C# dilinde veriler kullanım amaçlarına göre farklı türlere ayrılır:

- **Sayısal Türler:** Tam sayılar için `int`, `long`; ondalıklı sayılar için `float`, `double` ve hassas hesaplamalar için `decimal` kullanılır.
- **Metinsel Türler:** Tek bir karakter için `char`, metin blokları için `string` tercih edilir.
- **Mantıksal Tür:** Sadece `true` (doğru) veya `false` (yanlış) değerlerini alan `bool` türüdür.

3. Veri Türü Dönüşümleri

Farklı türdeki verilerin birbirine atanması sürecidir ve görsel programlamada (özellikle `TextBox` verilerinde) hayati önem taşır.

- **Bilinçsiz (Implicit) Dönüşüm:** Küçük veri türünün büyük türe otomatik olarak atanmasıdır; veri kaybı riski yoktur.
- **Bilinçli (Explicit) Dönüşüm:** Büyük veri türünün küçük türe kullanıcı tarafından zorlanarak atanmasıdır; veri kaybı riski taşır ve `cast` operatörü `( )` ile yapılır.
- **Helper Sınıflar:** Metinsel ifadeleri sayıya çevirmek için `Convert` sınıfı (Örn: `Convert.ToInt32( )` veya `Parse( )` metodları kullanılır.

4. Diziler (Arrays)

Aynı veri tipindeki birden fazla veriyi tek bir isim altında toplamak için kullanılır.

- **Tanımlama:** Köşeli parantez `[ ]` kullanılarak tanımlanır ve bellekte ne kadar yer kaplayacağı `new` anahtar kelimesiyle belirtilir.
- **İndis Yapısı:** Dizinin ilk elemanına her zaman **0 (sıfır)** indisi ile erişilir.
- **Dizi Metodları:** `Array.Sort( )` ile sıralama, `Array.Clear( )` ile temizleme ve `Array.Reverse( )` ile ters çevirme işlemleri yapılabilir.
- **Dinamik Diziler (ArrayList):** Boyutu önceden bilinmeyen ve farklı türde verileri bir arada tutabilen esnek dizi yapılarıdır.

7.ÜNİTE - İÇERİK, DOSYA VE KLASÖR İŞLEMLERİ

Windows dosya sisteminde dosya ve klasör oluşturma, silme, taşıma gibi temel işlemlerin yanı sıra, dosyaların içeriğine erişme ve disk bilgilerini listeleme yöntemleri ele alınmıştır. Tüm bu işlemler için .NET içerisindeki `System.IO` kütüphanesi temel alınır.

1. Dosya ve Klasör Yönetimi

Dosya sistemi üzerinde işlem yapmak için kullanılan iki ana yaklaşım (statik ve nesne tabanlı) bulunmaktadır.

- **File ve Directory Sınıfları:** Statik sınıflardır. Tek seferlik işlemler (Örn: `File.Create()`, `Directory.Exists()`) için hızlı ve pratiktir.
- **FileInfo ve DirectoryInfo Sınıfları:** Nesne tabanlı sınıflardır. Bir dosya veya klasör üzerinde birden fazla işlem yapılacaksa (Örn: hem adını değiştirip hem boyutunu okumak) performans avantajı sağlar.
- **Path Sınıfı:** Dosya yollarını (uzantı alma, tam yol birleştirme vb.) güvenli bir şekilde yönetmek için kullanılır.

2. Dosya İçeriği Okuma ve Yazma

Metin tabanlı veya bayt düzeyindeki verilerin işlenmesi için farklı "Stream" (Akış) sınıfları kullanılır.

- **StreamReader ve StreamWriter:** Metin dosyalarıyla (.txt vb.) çalışırken en sık kullanılan sınıflardır. Karakter bazlı okuma ve yazma yaparlar.
- **FileStream:** Dosyalar üzerinde bayt düzeyinde (resim, video veya ham veri) işlem yapmak için kullanılır. Daha düşük seviyeli ve esnek bir kontrol sağlar.
- **MemoryStream:** Verileri fiziksel bir disk yerine doğrudan RAM üzerinde (bellekte) geçici olarak depolamak ve işlemek için kullanılır.

3. Sürücü ve Yol İşlemleri

Sistemdeki depolama birimlerine erişim ve dosya konumlarının yönetimi ile ilgili işlevlerdir.

- **DriveInfo:** Bilgisayardaki diskleri (C:, D: vb.) listeler; disklerin toplam kapasitesini, boş alanını ve formatını öğrenmeyi sağlar.

- **Exists Kontrolü:** Herhangi bir dosya veya klasör üzerinde işlem yapmadan önce `Exists()` metodu ile varlığının kontrol edilmesi, uygulama hatalarını (Exception) önlemek için kritiktir.

4. Dosya İzleme (FileSystemWatcher)

Bu özel sınıf, belirli bir klasördeki değişiklikleri (yeni dosya eklenmesi, bir dosyanın silinmesi veya adının değişmesi) anlık olarak izlemek ve bu durum gerçekleştiğinde olay (event) fırlatmak için kullanılır.

- **Kritik İşlemler:**
 - **Copy / Move:** Dosyaları kopyalamak veya yerini değiştirmek için kullanılır.
 - **Delete:** Dosya veya klasörleri kalıcı olarak siler.
 - **GetFiles / GetDirectories:** Bir klasörün içeriğini listelemek için dizi (array) döndüren metodlardır.

8.ÜNİTE - UYGULAMANIN DAĞITIMI VE YAYIMLANMASI

Geliştirilen bir yazılımın hedef kitleye ulaştırılması için paketlenmesi, kurulum dosyalarının oluşturulması ve gerekli kütüphanelerin hazır edilmesi gerekir. Bu süreçte hata ayıklama modundan çıkış ve farklı yayımlama yöntemleri kritik rol oynar.

1. Derleme Modları: Debug ve Release

Yazılımın geliştirilme aşaması ile sunum aşaması arasındaki temel farktır.

- **Debug (Hata Ayıklama) Modu:** Yazılım geliştirilirken kullanılır ve kodun satır satır izlenmesine olanak sağlayan ek dosyalar (PDB) oluşturur. Çalışma hızı daha düşüktür ve dosya boyutu daha büyüktür.
- **Release (Yayımlama) Modu:** Son kullanıcıya sunulacak sürümdür. Kod optimize edilir, hata ayıklama bilgileri temizlenir ve uygulama daha hızlı çalışır.

2. Yayımlama Yöntemleri

Visual Studio, uygulamaları dağıtmak için farklı senaryolara uygun seçenekler sunar:

- **Klasöre Yayımlama:** Uygulamanın tüm dosyalarını yerel bir klasöre veya ağ paylaşımına kopyalar. En basit dağıtım yöntemidir.
- **ClickOnce Dağıtım:** Uygulamanın bir web sayfası veya ağ üzerinden kurulmasını sağlar. En büyük avantajı, uygulama güncellendiğinde kullanıcıların bilgisayarındaki sürümün otomatik olarak güncellenmesidir.
- **MSI Installer (Kurulum Paketi):** Kullanıcının alışık olduğu ".exe" veya ".msi" uzantılı kurulum sihirbazlarını oluşturur. Programın "Program Ekle/Kaldır" listesine eklenmesini sağlar.

3. Dağıtım Stratejileri

Uygulamanın çalışması için gerekli olan .NET çalışma zamanının (Runtime) nasıl sunulacağı ile ilgilidir:

- **Framework-Dependent (Çerçeveye Bağımlı):** Uygulama dosyaları küçüktür ancak hedef bilgisayarda uygun .NET sürümünün önceden kurulu olması gerekir.
- **Self-Contained (Bağımsız/Kendinden Kapsamlı):** .NET çalışma zamanı kütüphaneleri uygulamanın içine gömülür. Uygulama boyutu büyür ancak hedef bilgisayarda hiçbir ön kurulum gerekmeden çalışabilir.

4. Dağıtım Öncesi Kritik Kontroller

- **Bağımlılıklar:** Uygulamanın kullandığı harici kütüphanelerin (DLL), resimlerin veya veri tabanı dosyalarının paket içeriğine dahil edildiğinden emin olunmalıdır.
- **Hedef Platform:** Uygulamanın 32-bit (x86) mi yoksa 64-bit (x64) mi çalışacağı "Build" ayarlarından doğru seçilmelidir.
- **İkon ve Sürüm Bilgisi:** Profesyonel bir görünüm için uygulama simgesi (.ico) eklenmeli ve proje özelliklerinden sürüm numarası (Örn: 1.0.0) güncellenmelidir.